

13 — Operations & Troubleshooting

Audience: whoever is on call. **Companion docs:** [10 Upgrade & rollback](#) · [09 Backup & restore](#) · [02 Deployment](#)

1. First ninety seconds

```
cd ~/aug_11          # $PRISM_ROOT
./prism-deploy.sh status # what is live, what can be rolled back to
./prism-deploy.sh verify # health-check both doors

cd prism
docker compose -f deploy/compose/docker-compose.yml ps
docker compose -f deploy/compose/docker-compose.yml logs --tail=100 gateway register
```

`verify` is not a formality — it checks **two** things a naive check misses:

```
curl -sf http://127.0.0.1:$PORT/healthz # the GATEWAY lane
curl -sf http://127.0.0.1:$PORT/ui/    # the UI lane, a DIFFERENT upstream
```

A stack can be entirely healthy on the first and 502 on the second. That is the single commonest production incident here — §3.

2. Health endpoints

Endpoint	Answers
<code>GET /healthz</code>	The gateway is up (proxied by nginx on :80 and :443)
<code>GET /readyz</code>	The gateway's dependencies are ready
<code>GET /ui/</code>	The static UI container is reachable through the edge
Container healthchecks	<code>postgres</code> (<code>pg_isready</code>), <code>minio</code> (<code>mc ready</code>), and each service's own

```
# Anything not healthy?
docker ps --filter "label=com.docker.compose.project=compose" \
  --format '{{.Names}}\t{{.Status}}' | grep -iv healthy
```

3. The 502 that looks like a healthy stack

Symptom. Every page returns 502. `docker compose ps` shows everything healthy. The nginx log reads:

```
connect() failed (111: Connection refused) while connecting to upstream,
upstream: "http://172.18.0.4:80/ui/"
```

...and `172.18.0.4` belongs to no running container.

Cause. `deploy/nginx/nginx.conf` declares static upstreams:

```
upstream gateway { server gateway:8000; keepalive 32; }
upstream ui      { server ui:80;          keepalive 8; }
```

nginx resolves those names **once, at worker start**. Recreate `ui` or `gateway` and Docker hands them new addresses — but nginx's own image did not change, so it was never recreated, and it keeps dialling the old ones.

Fix.

```
docker compose -f deploy/compose/docker-compose.yml exec nginx nginx -s reload
# if the reload is refused:
docker compose -f deploy/compose/docker-compose.yml restart nginx
```

Prevention. `prism-deploy.sh` calls `reload_edge()` after every swap. If you recreate a container **by hand**, reload nginx yourself. `/dex/` and `/machine/v1/internal/` already use request-time DNS (`resolver 127.0.0.11 valid=10s`) and are immune.

4. Symptom → cause → fix

Platform-wide

Symptom	Likely cause	Fix
Every page 502	Stale nginx upstream	§3
Everything 401	<code>REQUIRE_AUTH</code> on with no reachable issuer — usually the <code>sso</code> profile was not passed	Bring the stack up with <code>--profile sso</code>
Everything 503	Access down while <code>ONLINE_REVALIDATION</code> is on	<code>docker compose logs access</code> ; it fails closed by design
TLS errors / edge will not start	Certificate missing or unreadable	Check <code>deploy/nginx/certs/</code> ; restore from the secrets snapshot
Slow everything	Postgres under load, or STT saturating the CPU	<code>docker stats</code> ; see §6
"no space left on device"	Disk	§7

Data plane

Symptom	Cause	Fix
403 on a write the user expects	Scope — the row is not theirs	Check <code>line_assignments</code> ; the matrix may be right
409 on save	Optimistic concurrency — someone else saved first	Re-read and re-apply. Working as designed
422 on a stage change	Illegal transition, or stage-mandatory data missing	The message names the field
A grid column is blank	A join could not resolve	<code>nameResolver</code> fails soft; check the row has <code>entity_id</code> or <code>deal_id</code>
An imported row is missing	Quarantined	Masters → Reconciliation
Database looks empty after an upgrade	Compose project name drifted → new, empty volumes	<code>docker volume ls</code> ; the data is in <code>compose_pgdata</code> . This is why the project name is pinned

Workflows

Symptom	Cause	Fix
Approval did nothing	Worker down — starts are accepted and queue silently	<code>docker compose ps workflows</code> ; restart it
Approval did nothing, worker is up	The signal had no durable decision record and was discarded by design	Check <code>workflow_decisions</code> ; approve through the UI, not by signalling Temporal
Activity retrying forever	A <i>refusal</i> classified as retryable	Fix the error classification (05 §5)
Temporal UI shows opaque payloads	Payload encryption on	Expected, not a fault

VocX

Symptom	Cause	Fix
"VocX did not answer in time"	STT exceeded the 240 s budget, usually queued	<code>docker compose logs stt</code> ; check <code>nproc</code> vs <code>STT_CPU_THREADS</code> ; add a replica
Recording stops at 3 minutes	<code>MAX_SECONDS</code> — by design	Raise <code>VITE_VOXC_MAX_SECONDS</code> , rebuild <code>ui</code> , reload nginx
413 on capture	Over 25 MB, or over nginx's 64 m	Shorter clip, or raise both
STT will not start	Baked model ≠ <code>STT_MODEL_SIZE</code>	Deliberate fail-fast; rebuild with the matching <code>ARG</code>
Google notes not written	Client secret absent, or <code>redirect_uri_mismatch</code>	Check the mount and the exact redirect URI

Notifications

Symptom	Cause	Fix
No emails	Notifier down, or SMTP misconfigured	<code>docker compose logs notifier</code> ; check <code>notification_deliveries</code> for stuck rows
Duplicate emails	Redrive without a claim	Check the delivery-claim endpoints

5. Logs

```
DC="docker compose -f deploy/compose/docker-compose.yml"
```

```
$DC logs -f gateway           # authorization decisions, routing, upstream errors
$DC logs -f register          # the book: writes, refusals, policy violations
$DC logs -f workflows         # the worker
$DC logs -f stt               # transcription timings
$DC logs -f nginx             # the edge – 502s live here
$DC logs --since 30m --tail 200 register gateway
```

Correlating a request across services

nginx stamps `X-Request-ID` on every request and echoes it back to the browser. It travels through the gateway to every downstream service, so:

```
$DC logs --since 1h | grep '<request-id>'
```

gives you one request's whole path. Ask a reporting user for the `X-Request-ID` from their browser's network tab.

Log volume is capped

Every service uses the `json-file` driver with **10 MB × 5 files = 50 MB per container**.

"the json-file driver grows without limit by default — one chatty or error-looping container can fill the VM disk over weeks and take the whole stack down, Postgres included."

6. Resource pressure

```
docker stats --no-stream
nproc
free -h
df -h
```

Component	Normal	Under stress
stt	Idle near zero; a decode pins its threads	Sustained 100% ⇒ captures are queuing
postgres	Low CPU, RAM for cache	High CPU ⇒ a missing index or a large export
register	Modest	Spikes on import/export
gateway / atlas / pulse / vocx	Small	—
workflows	Small, but must be running	—

If STT is the bottleneck

1. `nproc` on the host.
2. Compose has **no CPU limit** on `stt`. Set `STT_CPU_THREADS` to the cores the container may really use — `0` is often right on compose; `2` is right on Helm because the pod has `limits.cpu: 2`.
3. Still tight? **Add STT replicas**. Stateless, model in the image — it is the one service where replicas directly buy throughput.
4. Only as a last resort drop `STT_MODEL_SIZE`. That is the one knob that costs accuracy.

7. Disk

```
df -h
docker system df
du -sh ~/aug_11/releases/* ~/aug_11/backups/*
```

Reclaim, in order of safety:

```
docker image prune          # dangling only – the prism-rollback/* tags are TAGGED and
safe
docker builder prune        # build cache
rm -rf ~/aug_11/releases/failed-* # inspected failed releases
# old dumps – but keep enough to restore from
ls -lt ~/aug_11/backups/db-*.sql.gz | tail -n +11 | xargs -r rm -f
```

Never `docker system prune -a --volumes`. That removes the volumes holding the book.

`prism-deploy.sh` prunes automatically: `PRISM_KEEP_RELEASES` (default 3, **never** the rollback target) and `PRISM_KEEP_BACKUPS` (default 10). It refuses to start an upgrade with less than `PRISM_MIN_FREE_GB` (default 8 GB) free.

8. Restarting things safely

```
DC="docker compose -f deploy/compose/docker-compose.yml"

$DC restart register          # a single service...
$DC exec nginx nginx -s reload # ...then ALWAYS reload the edge

$DC restart workflows         # safe: in-flight workflows resume from history
$DC restart nginx             # safe: brief connection reset

$DC stop postgres             # everything fails while it is down
```

Rules:

- **Never** `docker compose down -v`. It removes volumes. `prism-deploy.sh` does not even use `down`.
- **After recreating any service, reload nginx.**
- Restarting the **workflows worker** is safe — Temporal resumes from history. Restarting it is often the right first move when approvals are stalled.

9. Routine checks

Daily

- ☐ `./prism-deploy.sh verify`
- ☐ No container in a restart loop: `docker ps --format '{{.Names}}\t{{.Status}}'`
- ☐ `df -h` — headroom

Weekly

- ☐ Newest DB dump < 24 h old and > 100 KB
- ☐ `pgbackup` still running
- ☐ Scan gateway/register logs for repeated 5xx
- ☐ `notification_deliveries` not accumulating unsent rows

Monthly

- ☐ TLS certificate expiry
- ☐ `docker system df` and reclaim
- ☐ Review `access_audit` for unexpected grants

Quarterly

- ☐ Restore drill (09 §6)
- ☐ Confirm the secrets snapshot opens and holds all three paths

10. Escalation notes

Before escalating, collect:

1. `./prism-deploy.sh status` output
2. `docker compose ps`
3. Logs for the failing service, `--since` the first report
4. The `X-Request-ID` of a failing request
5. Whether an upgrade happened recently, and the `backups/deploy-<stamp>.log`

If the platform is down and the cause is not obvious within ten minutes and there **was** a recent upgrade:

```
./prism-deploy.sh rollback
```

Rollback restores code and images, not data. It is the cheapest way to get the desk working again, and it keeps the failed tree at `releases/failed-<stamp>` for diagnosis afterwards.

11. Things that look like faults and are not

Worth knowing before you spend an hour on one.

Observation	Why it is correct
409 on a save	Optimistic concurrency preventing a lost update
A stage change refused with 422	The lifecycle graph, or stage-mandatory data
A signal to Temporal did nothing	Signals are untrusted until a durable decision record confirms them
Access down ⇒ deletes fail with 503	Sensitive operations fail closed on purpose
Temporal UI payloads unreadable	Payload encryption is on
A Deal Analyst owns a lead they cannot see	The view matrix gives them no Leads access
Three VocX captures about one new company created three leads	Three opportunities, filed separately
The recording stopped at 3 minutes	The configured cap; the clip so far was kept
A grid column is blank rather than erroring	<code>nameResolver</code> fails soft — an outage never fails the grid
An unknown query filter returns 400	Filters are whitelisted; silently ignoring one would widen a result set the caller believed was narrow